

03 — Spécification API

Swagger/OpenAPI : non trouvé dans le code actuel. Cette doc est générée à partir de `API/route/*.go` et `API/admin/*.go`. Base URL locale : `http://localhost:8081` — Prod : `https://upcycleconnect.pro/api` (Nginx retire le préfixe `/api`). Auth : header `Authorization: Bearer <token JWT>` sur toutes les routes protégées. Toutes les routes ont un `OPTIONS` (CORS préflight).

Convention de sécurité

- **Public** : login, inscription, mot de passe oublié, listes publiques d'annonces.
- **Token requis** : `VerifyTokenMiddleware` (`API/auth/jwt.go`).
- **Rôle requis** : `VerifyRoleMiddleware(handler, "Salarié", "Administrateur")` etc.

Authentification — `API/route/auth.go` , `API/route/users.go` → `API/admin/users.go`

Méthode	Route	Rôle	Handler	Paramètres	Réponse	Erreurs
POST	<code>/admin/login</code>	public	<code>admin.Login</code>	JSON <code>{email, mot_de_passe}</code>	<code>{token, id, role, prenom, score, tutorielVu, validation}</code>	401 email/mdp incorrect
POST	<code>/auth/inscription</code>	public	<code>admin.Inscription</code>	JSON user	<code>{message}</code> 201	500 email déjà pris
POST	<code>/auth/check-email</code>	public	—	<code>{email}</code>	dispo	—
POST	<code>/auth/forgot-password</code>	public	—	<code>{email}</code>	envoi lien	—
POST	<code>/auth/reset-password</code>	public	—	<code>{token, password}</code>	ok	400 token invalide

Exemple :

```
curl -X POST http://localhost:8081/admin/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@upcycle.fr","mot_de_passe":"motdepasse"}'
```

Utilisateurs (admin) — `API/route/users.go` → `API/admin/users.go`

Méthode	Route	Rôle	Paramètres	Réponse
GET	<code>/admin/users</code>	Salarié/Admin	—	<code>[User]</code>
GET	<code>/admin/users/{id}</code>	token	id (path)	User
GET	<code>/admin/users/search?search=</code>	token	query	<code>[User]</code>

Méthode	Route	Rôle	Paramètres	Réponse
GET	/admin/users/role/{role}	token	role (path)	[User]
POST	/admin/users/add	token	JSON user	{message, id}
PUT	/admin/users/modify/{id}	token	JSON	200
DELETE	/admin/users/delete/{id}	token	id	utilisateur suppr
PUT	/admin/users/validate/{id}	token	id	{message}
PUT	/admin/users/refuse/{id}	token	{motif}	{message}
PUT	/admin/users/ban/{id}	token	id	{message}

Exemple :

```
curl http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN"
```

Annonces — [API/route/annonces.go](#) → [API/admin/annonce.go](#)

Méthode	Route	Rôle	Paramètres	Réponse
GET	/api/annonces/all?id=	public	id user (query)	[Annonce] (jointes catégorie/user)
GET	/mes-annonces?id=	token	id user	[Annonce]
GET	/admin/annonces	token	—	[Annonce] (toutes)
GET	/admin/annonces/search?search=	token	query	[Annonce]
POST	/admin/annonces/add	token	multipart (titre, description, prix, id_categorie, image...)	201
PUT	/admin/annonces/modify/{id}	token	multipart	200
PUT	/admin/annonces/validate/{id}	token	id	{message}
PUT	/admin/annonces/refuse/{id}	token	id	{message}
DELETE	/admin/annonces/delete/{id}	token	id	{message}
POST	/api/payment-annonce	token	JSON paiement	session Stripe

Catégories — [API/route/categories.go](#) → [API/admin/categorie.go](#)

Méthode	Route	Rôle	Réponse
GET	/admin/categories	token	[{id, libelle}]
POST	/admin/categories/add	token	{libelle} → ok
DELETE	/admin/categories/delete/{id}	token	ok

Conteneurs / casiers — [API/route/logistique.go](#) → [API/admin/box.go](#)

Méthode	Route	Rôle	Réponse
GET	/api/admin/conteneurs	token	[Conteneur] (avec total_boxes)
GET	/api/admin/conteneur/{id}/boxes	token	[Box]
POST	/api/admin/conteneur/create	token	{nom, adresse, nombre_de_boxes}
POST	/api/admin/box/add	token	{id_conteneur, taille}
PUT	/api/admin/box/update	token	{box_id, statut}
POST	/api/box/reserve /deposit /collect	token	flux dépôt/retrait
GET	/api/user/boxes?user_id=	token	dépôts de l'utilisateur
GET	/api/user/pickups/{id}	token	retraits

Événements — API/route/evenements.go → API/admin/event.go

Méthode	Route	Rôle	Paramètres	Réponse
GET	/admin/evenements	token	—	[Evenement]
POST	/admin/evenements/add	token	multipart (titre, type, tarif, image, plan_pdf, ressources...)	ok (⚠ exige stripe_account_id du salarié)
PUT	/admin/evenements/{id}	token	multipart	modif
PUT	/admin/evenements/validate/{id} /refuse/{id}	token	id	{message}
POST	/admin/evenements/inscription /desinscription	token	{id_user, id_event}	ok
POST	/api/web/checkout/evenement	token	{id_user, id_event}	{checkout_url} (Stripe)
GET	/admin/evenements/inscrits/{id}	token	id event	liste inscrits
GET	/admin/evenements/ressources/{id}	token	id event	ressources PDF

Finance — API/route/finance.go → API/admin/orderReq.go

Méthode	Route	Rôle	Réponse
GET	/admin/finance/overview	token	{volumeMois, revenuMois}
GET	/admin/finance/transactions	token	[Transaction]
GET	/api/user/payment-history	token	historique

Eco / Score — API/route/divers.go → API/admin/annonce.go

Méthode	Route	Réponse
GET	/api/user/stats?user_id=	{score, objets_donnes, dechets_evites}
GET	/api/user/ecostats?user_id=	idem

Pro / abonnements — API/route/pro.go → API/admin/stripe.go , projet.go

Méthode	Route	Réponse
POST	/api/pro/subscribe /upgrade /cancel /portal	flux abonnement Stripe
GET	/api/pro/sync /invoices /projets /etapes	données pro
POST	/api/pro/projets/create , /api/pro/etapes/create	création
POST	/api/pro/annonces/sponsor	sponsoriser une annonce

Forum / Chat — API/route/forum.go , API/route/chat.go

Méthode	Route	Réponse
GET/POST	/user/forums	sujets
GET/POST	/user/forums/messages?topic_id=	messages (visibles = est_modere=0)
PUT	/admin/forum/messages/moderate/{id}	modération
GET	/ws/chat	WebSocket messagerie
GET	/api/chat/conversations /api/chat/history	chat

Notifications — API/route/notifications.go → API/admin/notifications.go

Méthode	Route	Réponse
POST	/admin/notifications/send	{nombre} envoyées
GET	/admin/notifications/user/{id}	[notif]
POST	/admin/notifications/user/{id}/read	marque lu

Traductions — API/route/traductions.go

Méthode	Route	Réponse
GET	/api/translations?lang=fr	{clé: valeur}
GET	/api/languages	[{code, name}]
POST	/admin/translations/add	import JSON

Documents / PDF — API/route/documents.go → API/admin/document.go

Méthode	Route	Réponse
GET	/admin/documents	[{type_doc, url_pdf, ...}]
—	GET /uploads/documents/...	fichier PDF (FileServer, API/route/auth.go)

- `GET /uploads/*` → `http.FileServer` sur `UPLOAD_DIR` (`/app/uploads`) : images annonces/articles/events/formations + PDF documents.